

PSYC 51.17: Models of Language and Communication - Week 3

Classic Embeddings: LSA & LDA¹

Lecture 9: The Foundations of Distributional Semantics

PSYC 51.17: Models of Language and Communication - Week 3

Winter 2026

Winter 2026

Today's Lecture

1. **The Distributional Hypothesis**
2. **Vector Space Models**
3. **Latent Semantic Analysis (LSA)**
4. **Latent Dirichlet Allocation (LDA)**
5. **Modern Topic Modeling: BERTopic**
6. **Evaluation & Applications**

Goal: Understand how classic methods represent meaning through co-occurrence

Winter 2026

The Fundamental Question

How do we represent the meaning of words in a way that computers can understand?

\pause

Traditional Approach:

- Dictionaries
- Taxonomies (WordNet)
- Manual feature engineering

Distributional Approach:

- Learn from data
- Context-based
- Scalable

The Distributional Hypothesis

"You shall know a word by the company it keeps"

--- J.R. Firth (1957)

\pause

Key Idea:

Words that appear in similar contexts tend to have similar meanings.

Example

- "The **cat** sat on the mat"
- "The **dog** sat on the mat"

From Words to Vectors

Goal: Represent each word as a point in high-dimensional space

Concrete Example

Consider a tiny corpus with 3 documents:

- Doc 1: "The cat sat on the mat"
- Doc 2: "The dog ran in the park"
- Doc 3: "The cat and dog played"

Word-context co-occurrence:

Word	appears with "the"	appears with "sat"	appears with "ran"

Building a Term-Document Matrix

Step 1: Count how often each word appears in each document

cat	0	4	1	0
compute r	0	0	0	5
code	0	0	0	3
animal	2	3	2	0

6

TF-IDF Weighting

Term Frequency - Inverse Document Frequency

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Term Frequency (TF):

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_{t'} \text{count}(t', d)}$$

How often does term t appear
in document d ?

Rewards frequent terms in

Inverse Document Frequency (IDF):

$$\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$$

How rare is term t across all
documents?

TF-IDF: Worked Example

Corpus: 3 documents, 1000 total documents in collection

Document	Text
Doc 1	"machine learning is great"
Doc 2	"deep learning models"
Doc 3	"machine translation works"

Latent Semantic Analysis (LSA)

The OG of semantic embeddings (1990)

Algorithm:

1. Build term-document matrix X
2. Apply TF-IDF weighting
3. Perform SVD: $X = U\Sigma V^T$
4. Keep top k dimensions

SVD Decomposition:

1	$X = U \times \Sigma \times V^T$
---	----------------------------------

2	$(m \times n) \quad (m \times k)$ $(k \times k) \quad (k \times n)$
---	--

3	words/docs
---	------------

LSA: Step-by-Step Worked Example

Mini corpus (4 words × 3 documents):

```
1 | Doc1 Doc2 Doc3
```

Step 1: Apply SVD $X = U\Sigma V^T$

Step 2: Keep top 2 dimensions (k=2)

```
1 | from sklearn.decomposition import TruncatedSVD
```

Result: "pet" and "animal" end up close together in
2D space!

Winter 2026

LSA: How It Works

Example: Discovering latent topics

Original Dimensions:

- 50,000 terms
- 10,000 documents
- Matrix: 50k × 10k
- Sparse and noisy

After LSA:

- 100-300 latent dimensions
- Dense representation
- Captures semantic patterns

Discovered Topics (example):

Topic 1: Technology

computer (0.8), software (0.7), code (0.6), algorithm (0.5)

Topic 2: Animals

dog (0.9), cat (0.8), pet (0.7), animal (0.6)

Topic 3: Sports

game (0.8), team (0.7), player (0.6), win

LSA Limitations

Computational Issues:

- SVD is expensive: $O(\min(nm^2, n^2m))$
- Hard to update with new documents
- Memory intensive for large corpora

Modeling Issues:

- Linear relationships only
- Bag-of-words (loses word order)
- No polysemy handling

Example Problem: Polysemy

Word: "bank"

1. Financial institution
2. River bank
3. Banking (airplane maneuver)

LSA gives for all meanings!

But...

- Still useful for many tasks
- Foundation for modern methods

Latent Dirichlet Allocation (LDA)

A probabilistic approach to topic modeling

What if we model documents as probabilistic mixtures of topics?

Key Assumptions:

1. Each document is a mixture of topics
2. Each topic is a distribution

1	Topic 1 →	
	Topic 2 →	
	Topic 3 →	
	sports, game	13

LDA: The Generative Story

How LDA imagines documents are created:

Generative Process:

1. Choose number of topics K
2. For each topic k :
 - Draw word distribution $\phi_k \sim \text{Dir}(\beta)$
3. For each document d :
 - Draw topic distribution $\theta_d \sim \text{Dir}(\alpha)$

Key Parameters:

- α : Document-topic density
- Low $\alpha \rightarrow$ few topics per doc
- High $\alpha \rightarrow$ many topics per doc
- β : Topic-word density
- Low $\beta \rightarrow$ focused topics
- High $\beta \rightarrow$ general topics

LDA: Concrete Generative Example

Imagine generating a document about "tech pets":

```
1 | Step 1: Pick topic mixture for this document
2 |    $\theta_{\text{doc}} = [0.6 \text{ Tech}, 0.3 \text{ Animals}, 0.1 \text{ Sports}]$ 
3 |
4 | Step 2: For each word, sample a topic, then sample a word:
5 |
6 | Word 1: Sample topic  $\rightarrow$  Tech (60% chance)
7 |   Sample word from Tech  $\rightarrow$  "software"
```

LDA reverses this: Given documents, infer the topics!

LDA in Practice

```
1 from sklearn.decomposition import LatentDirichletAllocation
2 from sklearn.feature_extraction.text import CountVectorizer
3
4 # Prepare documents
5 documents = [
6     "The cat sat on the mat",
7     "The dog ran in the park",
8     "Machine learning is amazing",
9     # ... more documents
10 ]
11
12 # Create bag-of-words representation
13 vectorizer = CountVectorizer(max_features=1000, stop_words='english')
14 doc_term_matrix = vectorizer.fit_transform(documents)
15 vocab = vectorizer.get_feature_names_out()
16
```

Winter 2026

16

continued...

LDA in Practice

```
17 # Fit LDA model
18 lda = LatentDirichletAllocation(
19     n_components=10, # number of topics
20     random_state=42,
21     max_iter=50
22 )
23
24 doc_topics = lda.fit_transform(doc_term_matrix)
25
26 # Print top words per topic
27 for idx, topic in enumerate(lda.components_):
28     top_words = [vocab[i] for i in topic.argsort()[-10:]]
29     print(f"Topic {idx}: {' '.join(top_words)}")
```

Winter 2026

17

...continued

LDA Example: Topic Discovery

Example Topics from News Corpus:

Topic 1: Politics (20%)

government, election, president, vote, policy, senate,
congress, party

Topic 2: Sports (15%)

game, team, player, score, win, season, coach,
championship

Topic 3: Technology (25%)

software, computer, data, algorithm, system, internet,

Modern Topic Modeling: BERTopic

Combining neural embeddings with topic models

BERTopic Pipeline:

1. Embed documents with BERT
2. Reduce dimensionality with UMAP
3. Cluster with HDBSCAN
4. Generate topics via c-TF-IDF
5. Fine-tune with MaximalMarginalRelevance

Advantages:

- Leverages pre-trained models
- Better semantic understanding
- Automatic topic number detection
- Dynamic topic modeling

```
1 from bertopic import BERTopic
2 from sentence_transformers import
  SentenceTransformer
3
4 # Initialize
5 embedding_model =
6   SentenceTransformer(
7     "all-MiniLM-L6-v2"
8   )
```

continued...

Modern Topic Modeling: BERTopic

```
9 model = BERTopic(  
10     embedding_model=embedding_model,  
11     language="english",  
12     calculate_probabilities=True,  
13     verbose=True  
14 )  
15  
16 # Fit and transform
```

Winter 2026

Modern Topic Modeling: BERTopic

```
17 | topics, probs = model.fit_transform(  
18 |     documents  
19 | )  
20 |  
21 | # Get topic info  
22 | topic_info = model.get_topic_info()  
23 |  
24 | # Visualize
```

Winter 2026

Modern Topic Modeling: BERTopic

```
25 | fig = model.visualize_topics()
```

Reference: Grootendorst (2022) - "BERTopic: Neural topic modeling with a class-based TF-IDF procedure"

Winter 2026

LSA vs. LDA vs. BERTopic

Interpretability	Medium	High	High
Topic coherence	Medium	High	Very High
Computation	Fast	Medium	Slow
Scalability	Good	Good	Medium
Pre-training	No	No	Yes
Hyperparameters	Few	Many	Many
Topic number	Fixed	Fixed	Automatic

Recommendations

- **LSA**: Quick exploration, large-scale retrieval
- **LDA**: Interpretable topics, when you know topic count

Evaluating Topic Models

Intrinsic Metrics:

- **Perplexity:** Lower is better
- Measures likelihood
- Can be misleading!
- **Topic Coherence:** Higher is better
- Measures semantic similarity
- Better correlation with human judgment
- CV, UCI, UMass variants
- **Topic Diversity:**
- Unique words across topics
- Avoids redundant topics

Extrinsic Metrics:

- Document classification accuracy
- Information retrieval performance
- Clustering quality

Human Evaluation:

- Topic interpretability
- Word intruder detection
- Topic utility for task

Coherence Evaluation: Code Example

```
1 from gensim.models import LdaModel
2 from gensim.models.coherencemodel import CoherenceModel
3 from gensim.corpora import Dictionary
4
5 # Prepare corpus
6 texts = [doc.split() for doc in documents]
7 dictionary = Dictionary(texts)
8 corpus = [dictionary.doc2bow(text) for text in texts]
9
10 # Train LDA with different topic numbers
11 coherence_scores = []
12 for num_topics in [5, 10, 15, 20, 25]:
13     lda = LdaModel(corpus, num_topics=num_topics,
14                     id2word=dictionary, passes=10)
15
16 # Calculate coherence (C_V is recommended)
17 coherence = CoherenceModel(model=lda, texts=texts,
```

25

Winter 2026

continued...

Coherence Evaluation: Code Example

```
18 dictionary=dictionary,  
19 coherence='c_v')  
20 coherence_scores.append((num_topics, coherence.get_coherence()))  
21  
22 # Results: [(5, 0.42), (10, 0.51), (15, 0.48), (20, 0.45), (25, 0.41)]  
23 # Best: 10 topics with coherence 0.51
```

Rule of thumb: Higher coherence = more interpretable topics

Winter 2026

26

...continued

- Topic discovery
- Trend analysis

3. Text Mining

- Opinion mining
- Literature review
- Knowledge discovery

Application Example: Semantic Search with LSA

```
1 from sklearn.decomposition import TruncatedSVD
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 # Documents already vectorized with TF-IDF
5 # tfidf_matrix shape: (1000 docs, 5000 words)
6
7 # Apply LSA to reduce dimensions
8 lsa = TruncatedSVD(n_components=100)
9 doc_embeddings = lsa.fit_transform(tfidf_matrix)
10
11 # User query: "machine learning algorithms"
12 query_tfidf = vectorizer.transform(["machine learning algorithms"])
13 query_embedding = lsa.transform(query_tfidf)
14
15 # Find most similar documents
16 similarities = cosine_similarity(query_embedding, doc_embeddings)[0]
17 top_5_docs = similarities.argsort()[-5:][::-1]
```

28

Winter 2026

continued...

Application Example: Semantic Search with LSA

```
18  
19 print("Most relevant documents:")  
20 for idx in top_5_docs:  
21     print(f" Doc {idx}: similarity = {similarities[idx]:.3f}")
```

Key insight: LSA finds documents about "neural networks" and "deep learning" even though those exact words weren't in the query!

Winter 2026

29

...continued

Discussion Question

Can distributional semantics truly capture meaning?

Arguments For:

- Captures semantic similarity
- Works well in practice
- Aligns with linguistic theory
- Scalable to large corpora
- Unsupervised learning

Arguments Against:

- Text-only (no grounding)
- Missing embodied experience
- No common sense reasoning

2. Choosing Parameters:

- Start with 10-50 topics for LDA
- Use perplexity for model selection
- Validate with coherence scores
- Try multiple random seeds

3. Interpreting Results:

- Look at top 10-20 words per topic
- Examine representative documents
- Check for duplicate/junk topics

3. **LDA (2003):**

- Probabilistic topic modeling
- Documents as mixtures of topics
- Highly interpretable

4. **BERTopic (2022):**

- Harnad, S. (1990). "The symbol grounding problem"

Modern Extensions:

- Boleda, G. (2020). "Distributional Semantics and Linguistic Theory"
- Grootendorst, M. (2022). "BERTopic: Neural topic modeling with a class-based TF-IDF procedure"
- Grand et al. (2022). "Semantic projection recovers rich human knowledge of multiple object features"

Tools & Libraries:

Questions?

Next Lecture:

Word Embeddings: Word2Vec, GloVe, FastText

From count-based to prediction-based methods!

Winter 2026